University of Brescia, Italy
Machine Learning& Data Mining

Bayesian Learning

Alfonso E. Gerevini

Bayesian Learning

- Bayes Theorem
- MAP, ML hypotheses
- MAP learners
- Bayes optimal classifier
- Naive Bayes learner
- Example: Learning over text data

Two Roles for Bayesian Methods

(1) Provides practical learning algorithms:

- Bayesian learning: examples affect the **probability** that a hypothesis is correct
- Combine **prior knowledge** with observed data (prior probabilities on hypotheses and observable data)
- Predictions have **probabilities** (new instances classified by weighted combination of **multiple hypotheses**)
- But requires prior probabilities (often estimated from available data)

(2) Provides useful conceptual framework

- Provides "gold standard" for evaluating other learning algorithms

Basic Formulas for Probabilities

- *Product Rule*: probability $P(A \wedge B)$ of a conjunction of two events A and B:
$$P(A \wedge B) = P(A|B)P(B) = P(B|A)P(A)$$
- *Sum Rule*: probability of a disjunction of two events A and B:
$$P(A \vee B) = P(A) + P(B) - P(A \wedge B)$$
- *Theorem of total probability*: if events $A_1, \ldots, A_n$ are mutually exclusive with $\sum_{i=1}^{n} P(A_i) = 1$, then
$$P(B) = \sum_{i=1}^{n} P(B|A_i)P(A_i)$$

Bayes Theorem

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}$$

- $P(h)$ = prior probability of hypothesis h
- $P(D)$ = prior probability of training data D
- $P(h|D)$ = probability of h given D (posterior probability)
- $P(D|h)$ = probability of D given h

Choosing Hypotheses

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}$$

Generally we want the most probable hypothesis given D
Maximum a posteriori hypothesis h_{MAP}:

$$h_{MAP} = \arg\max_{h \in H} P(h|D) = \arg\max_{h \in H} \frac{P(D|h)P(h)}{P(D)} = \arg\max_{h \in H} P(D|h)P(h)$$

because $P(D)$ does not depend on h

If assume $P(h_i) = P(h_j)$ (uniform probability distribution over H), we can further simplify, and choose the *Maximum likelihood* (ML) hypothesis

$$h_{ML} = \arg\max_{h \in H} P(D|h)$$

Brute Force MAP Hypothesis Learner

1. For each hypothesis h in H, calculate the posterior probability

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}$$

2. Output the hypothesis h_{MAP} with the highest posterior probability

$$h_{MAP} = \underset{h \in H}{\operatorname{argmax}} P(h|D)$$

Brute Force MAP Concept Learning

Consider our usual concept learning task

- instance space X, hypothesis space H, training examples D
- consider the FindS learning algorithm (outputs most specific hypothesis from the version space $VS_{H,D}$)

What would Bayes rule produce as the MAP hypothesis?

Does *FindS* output a MAP hypothesis?

Brute Force MAP Concept Learning

Assume no noise in training data and target function in H

Assume fixed set of instances $\langle x_1, \ldots, x_m \rangle$

Assume D is the set of classifications $D = \langle c(x_1), \ldots, c(x_m) \rangle$

- Choose $P(D|h)$:

$$P(D|h) = \begin{cases} 1 & \text{if } h \text{ consistent with } D \text{ (because no noisy data)} \\ 0 & \text{otherwise} \end{cases}$$

- Choose $P(h)$ to be *uniform* distribution: $P(h) = \frac{1}{|H|}$ for all h in H

Brute Force MAP Concept Learning

Then,

$$P(D) = \sum_{h_i \in H} P(D|h_i)P(h_i) = \ldots = \frac{|VS_{H,D}|}{|H|}$$

$$P(h|D) = \begin{cases} \frac{1}{|VS_{H,D}|} & \text{if } h \text{ is consistent with } D \text{ (i.e. } h \in VS_{H,D}) \\ 0 & \text{otherwise} \end{cases}$$

$|VS_{H,D}|$ = number of different hypotheses consistent with D

Brute Force MAP Concept Learning

$$P(D) = \sum_{h_i \in H} P(D|h_i)P(h_i)$$

$$= \sum_{h_i \in VS_{H,D}} 1 \cdot \frac{1}{|H|} + \sum_{h_i \notin VS_{H,D}} 0 \cdot \frac{1}{|H|} = \sum_{h_i \in VS_{H,D}} 1 \cdot \frac{1}{|H|} = \frac{|VS_{H,D}|}{|H|}$$

$$P(h|D) = \frac{1 \cdot \frac{1}{|H|}}{P(D)} = \frac{1 \cdot \frac{1}{|H|}}{\frac{|VS_{H,D}|}{H}} = \frac{1}{|VS_{H,D}|} \text{ if } h \text{ consistent with } D$$

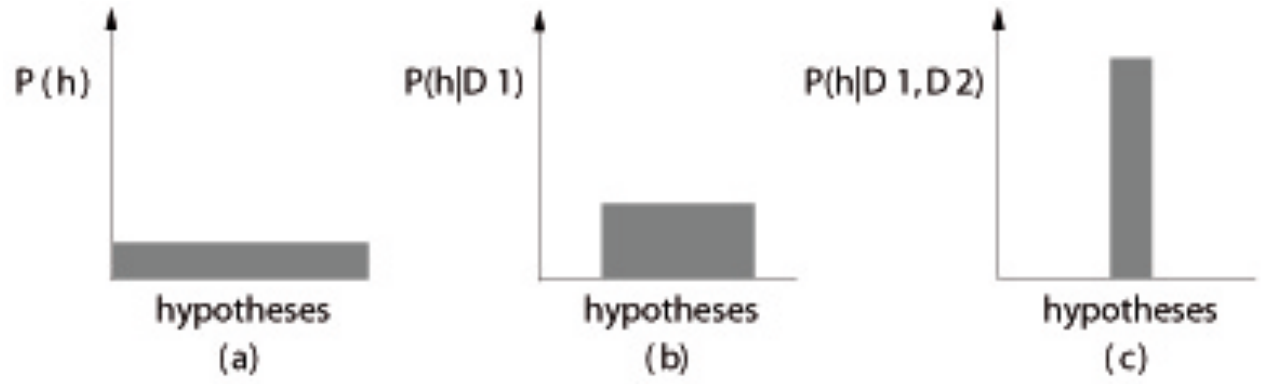
Relation to Concept Learning

Consistent learner: it outputs h with zero errors for training examples

- Every consistent hypothesis has posterior probability $\frac{1}{|VS_{H,D}|}$.
- Every consistent hypothesis is a MAP hypothesis.
- *Consistent learners* output a MAP hypothesis, if we assume uniform prior probabilities and deterministic, noise-free training data.
- *Consistent learners* can output MAP hypotheses also with different prior probability distributions.

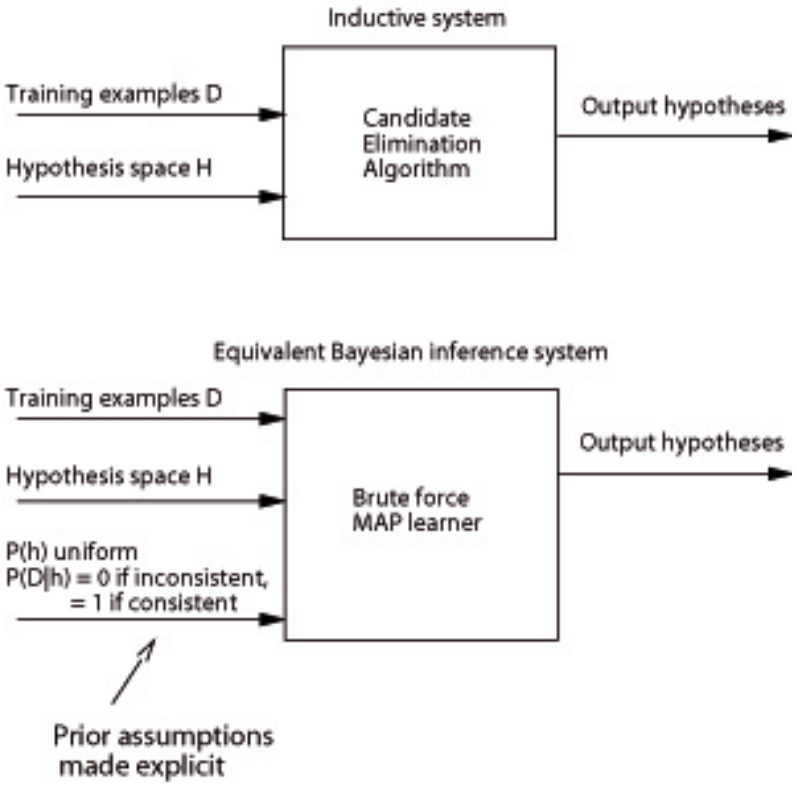
For example, FIND-S outputs a MAP hypothesis if we assume a probability distribution $P(h)$ that assigns $P(h_1) \geq P(h_2)$ if h_1 is more specific than h_2.

Evolution of Posterior Probabilities



Evolution of $P(h|D)$ with increasing training data. Version space reduces with new examples

Learning Algorithms vs. MAP Learners



Bayesian Learning Example 1

h_1:	10 %	100 % cherry	0 % lime candies
h_2:	20 %	75 % cherry	25 % lime candies
h_3:	40 %	50 % cherry	50 % lime candies
h_4:	20 %	25 % cherry	75 % lime candies
h_5:	10 %	0 % cherry	100 % lime candies

Assumption: bags contain a high, indefinite number of candies

Bayesian Learning Example 1

Observation

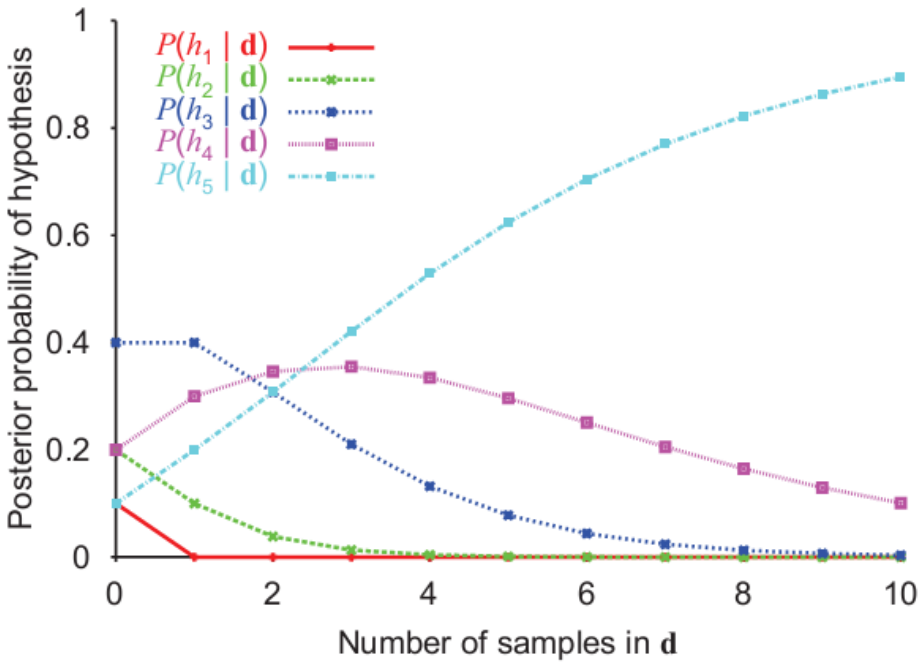
Candies drawn from a bag (unknown type):

- $\mathbf{d} = \{L, L, L, L, L, L, L, L, L, L\}$ (i.e., 10 lime candies)

Questions

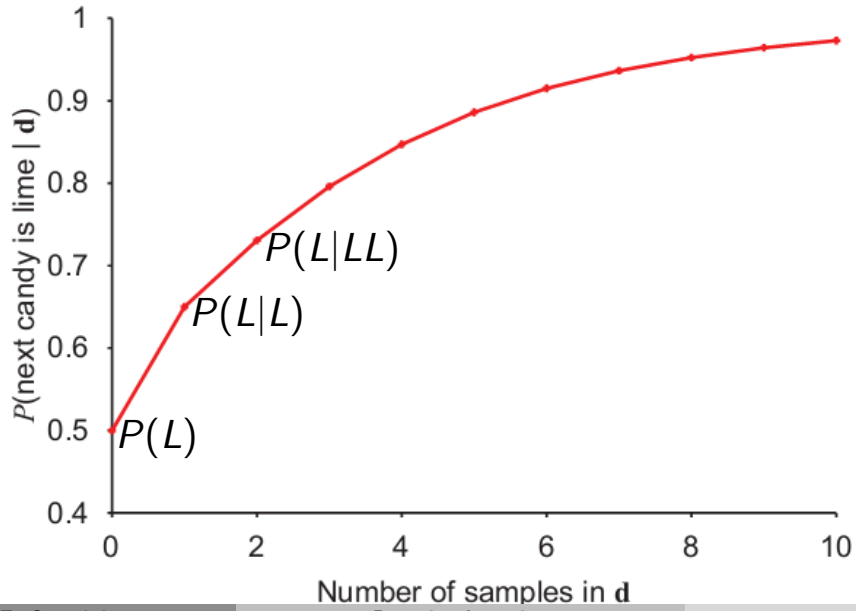
- What kind of bag is it (MAP hypothesis)?
- What flavour will the next candy be?

Bayesian Learning Example 1



Bayesian Learning Example 1

$P(L) = \sum_{h_i \in H} P(h_i) \cdot P(L|h_i) = 0.5$ $\mathbf{d} = L, L, L, L....$



Bayesian Learning Example 2

Consider a new manufacturer producing bags with an arbitrary choice of cherry/lime candies. $\theta \equiv \frac{nr.\ of\ cherry\ candies}{N} \in [0,1]$.

Continuous space for hypotheses: h_θ (= values of θ)

Data set (opened candies): $\mathbf{d} = \{c\ \text{cherries}, l\ \text{lime}\}$, $N = c + l$

If we assume all proportions of cherry candies are equally likely a priori, then ML approach is reasonable

- What is the ML hypothesis (i.e., value of θ)?

Bayesian Learning Example 2

$$h_{ML} = \underset{h_\theta}{\operatorname{argmax}}\, P(\mathbf{d}|h_\theta) = \underset{h_\theta}{\operatorname{argmax}}\, L(\mathbf{d}|h_\theta)$$

with $L(\mathbf{d}|h_\theta) = \log P(\mathbf{d}|h_\theta)$ - log useful because changes the product into a sum

$$P(\mathbf{d}|h_\theta) = \prod_{j=1\ldots N} P(d_j|h_\theta) = \theta^c \cdot (1-\theta)^l$$

$$L(\mathbf{d}|h_\theta) = c \log\theta + l \log(1-\theta)$$

Computation of θ maximizing $\log P(\mathbf{d}|h_\theta)$:

$$\frac{dL(\mathbf{d}|h_\theta)}{d\theta} = \frac{c}{\theta} - \frac{l}{1-\theta} = 0 \Rightarrow \frac{c(1-\theta) - l\theta}{\theta(1-\theta)} = 0$$

$$\Rightarrow\ c(1-\theta) - l\theta = 0 \Rightarrow\ \theta_{ML} = \frac{c}{c+l} = \frac{c}{N}$$

Most Probable Classification of New Instances

So far we have sought the most probable *hypothesis* given the data D (i.e., h_{MAP})

Given a new instance x, what is its most probable *classification*?

- Is $h_{MAP}(x)$ the most probable classification ?

Most Probable Classification of New Instances

Consider:

- Three possible hypotheses:
  $$P(h_1|D) = 0.4,\ P(h_2|D) = 0.3,\ P(h_3|D) = 0.3$$
- Given a new instance x,
  $$h_1(x) = \oplus,\ h_2(x) = \ominus,\ h_3(x) = \ominus$$
- What's most probable classification of x ?

Most Probable Classification of New Instances

Answer:

Since $h_{MAP} = h_1$, if we consider one hypothesis, the answer is $\oplus$

But is we consider ALL hypotheses, I predict $\oplus$ with prob 0.4 and $\ominus$ with prob $0.3 + 0.3 = 0.6$!

Bayes Optimal Classifier

More in general:

Consider classification of a new instance x assuming (target) values v_j in the set V

$$P(v_j|D) = \sum_{h_i \in H} P(v_j|h_i)P(h_i|D)$$

Note: in the previous examples $P(v_j|h_i) \in \{0, 1\}$
($h_i(x)$ gives $\oplus/\ominus$ with prob 1)
$P(v_j|h_i)$ is the probability that h_i predicts class v_j

Bayes optimal classification:

$$v^* = \arg\max_{v_j \in V} \sum_{h_i \in H} P(v_j|h_i)P(h_i|D)$$

Bayes Optimal Classifier

Example: classification of new instance x assuming values in $V = \{\oplus, \ominus\}$

$$\begin{aligned} P(h_1|D) = 0.4, \quad P(\ominus|h_1) = 0, \quad P(\oplus|h_1) = 1 \\ P(h_2|D) = 0.3, \quad P(\ominus|h_2) = 1, \quad P(\oplus|h_2) = 0 \\ P(h_3|D) = 0.3, \quad P(\ominus|h_3) = 1, \quad P(\oplus|h_3) = 0 \end{aligned}$$

therefore

$$\begin{aligned} P(\oplus|D) = \sum_{h_i \in H} P(\oplus|h_i)P(h_i|D) &= 0.4 \\ P(\ominus|D) = \sum_{h_i \in H} P(\ominus|h_i)P(h_i|D) &= 0.6 \end{aligned}$$

and

$$v^* = \arg\max_{v_j \in V} \sum_{h_i \in H} P(v_j|h_i)P(h_i|D) = \ominus$$

Bayes Optimal Classifier

- *Optimal learner*: no other classification method using the same hypothesis space and same prior knowledge can outperform this method on average.
- It maximizes the probability that the new instance is classified correctly, i.e., $\operatorname{argmax}_{v_j \in V} P(v_j|D)$.
- Predictions made can correspond to a hypothesis not contained in H!!

  New instances labelled $\operatorname{argmax}_{v_j \in V} P(v_j|D)$ which can correspond to *no specific hypothesis in H* because $P(v_j|D)$ is a (linear) combination of multiply hypotheses

Gibbs Classifier

Bayes optimal classifier provides best result, but can be expensive if many hypotheses.

Gibbs algorithm:

1. Choose one hypothesis at random, according to $P(h|D)$
2. Use this to classify new instance

Surprising fact: It works quite well!
Assume target concepts (target boolean functions) are drawn at random from H according to priors on H. Then:

$$Average[error_{Gibbs}] \leq 2Average[error_{BayesOptimal}]$$

Gibbs Classifier

Suppose correct, uniform prior distribution over H, then

- Pick any hypothesis from VS, with uniform probability
- Its expected error no worse than twice Bayes optimal

Naive Bayes Classifier

Along with decision trees, neural networks, nearest neighbourhood, **one of the most practical learning methods!!**

When to use

- Moderate or large training set available
- Attributes that describe instances are conditionally independent given classification

Successful applications:

- Diagnosis
- Classifying text documents

Conditional Independence

Definition: *X is* conditionally independent *of Y given Z if the probability distribution governing X is independent of the value of Y given the value of Z; that is, if*

$$(\forall x_i, y_j, z_k)\ P(X = x_i | Y = y_j, Z = z_k) = P(X = x_i | Z = z_k)$$

more compactly, we write

$$P(X|Y, Z) = P(X|Z)$$

Naive Bayes uses conditional independency to justify

$$P(X, Y|Z) = P(X|Y, Z)P(Y|Z) = P(X|Z)P(Y|Z)$$

Naive Bayes Classifier (no explicit hypothesis)

Assume $f : X \to V$ target function, where each instance x is described by attributes $\langle a_1, a_2 \dots a_n \rangle$, V is finite set, and d is set of instances (dataset).

Most probable value of $f(x)$ of new instance x given d is:

$$\begin{aligned} v_{MAP} &= \operatorname*{argmax}_{v_j \in V} P(v_j | a_1, a_2 \dots a_n, d) \\ v_{MAP} &= \operatorname*{argmax}_{v_j \in V} \frac{P(a_1, a_2 \dots a_n | v_j, d) P(v_j | d)}{P(a_1, a_2 \dots a_n | d)} \quad [\textit{Bayes theorem}] \\ &= \operatorname*{argmax}_{v_j \in V} P(a_1, a_2 \dots a_n | v_j, d) P(v_j | d) \quad [\textit{Denom. ind. from } v_j] \end{aligned}$$

Need of a **big** number of training instances to estimate these probs...

Naive Bayes Classifier

Naive Bayes assumption:

$$P(a_1, a_2 \dots a_n | v_j, d) = \prod_{i=1 \dots n} P(a_i | v_j, d)$$

which gives an *approximation* of v_{MAP}:

Naive Bayes classifier: $v_{NB} = \operatorname*{argmax}_{v_j \in V} P(v_j | d) \prod_{i=1 \dots n} P(a_i | v_j, d)$

Question: How do we estimate $P(v_j | d)$? and $P(a_i | v_j, d)$?

Easy answer: *count v_j and (v_j, a_i) frequencies in the training dataset*

Naive Bayes Algorithm

Naive_Bayes(d, x)

For each target value v_j

$\hat{P}(v_j|d) \leftarrow$ estimate $P(v_j|d)$

For each attribute value α_k of each attribute a_i

$\hat{P}(\alpha_k|v_j, d) \leftarrow$ estimate $P(\alpha_k|v_j, d)$

Classify_New_Instance($x = \langle \alpha_1, \ldots, \alpha_n \rangle$)

$$v_{NB} = \underset{v_j \in V}{\operatorname{argmax}} \hat{P}(v_j|d) \prod_{i=1\ldots n} \hat{P}(\alpha_i|v_j, d)$$

Notes:

- The set of $\hat{P}$s and the v_{NB} rule corresponds to the learned h.
- No search in the hypothesis space.

Naive Bayes: Example

Consider *PlayTennis* again, and new instance

$$\langle Outlook = sun, Temp = cool, Humid = high, Wind = strong \rangle$$

Want to compute:

$$v_{NB} = \underset{v_j \in V}{\operatorname{argmax}} P(v_j|d) \prod_{i=1\ldots n} P(\alpha_i|v_j, d)$$

Naive Bayes: Example

Estimation of conditional probabilities:

$$\hat{P}(v_j|d) = \frac{|\{V = v_j\}|}{|d|}$$

$$\hat{P}(PlayTennis = yes|d) = P(y|d) = 9/14 = 0.64$$
$$\hat{P}(PlayTennis = no|d) = P(n|d) = 5/14 = 0.36$$

$$\hat{P}(\alpha_i|v_j, d) = \frac{|\{A = \alpha_i, V = v_j\}|}{|\{V = v_j\}|}$$

$$\hat{P}(Wind = strong|PlayTennis = yes, d) = 3/9 = 0.33$$
$$\hat{P}(Wind = strong|PlayTennis = no, d) = 3/5 = 0.60$$

Naive Bayes: Example

Naive Bayes solution

$$v_{NB} = \underset{v_j \in V}{\operatorname{argmax}} \hat{P}(v_j|d) \prod_{i=1...n} \hat{P}(\alpha_i|v_j, d)$$

$j \in \{1, 2\}$, $v_1 = yes(y)$, $v_2 = no(n)$

$$\hat{P}(y|d)\ \hat{P}(sun|y, d)\ \hat{P}(cool|y, d)\ \hat{P}(high|y, d)\ \hat{P}(strong|y, d) = .005$$
$$\hat{P}(n|d)\ \hat{P}(sun|n, d)\ \hat{P}(cool|n, d)\ \hat{P}(high|n, d)\ \hat{P}(strong|n, d) = .021$$

$$\rightarrow v_{NB} = no$$

Naive Bayes Remarks

1. **Conditional independence assumption is often violated**

$$P(a_1, a_2 \ldots a_n | v_j) = \prod_i P(a_i | v_j)$$

...but it works surprisingly well anyway!

- Don't need estimated posteriors $\hat{P}(v_j|x, d)$ to be correct; need only that

$$\underset{v_j \in V}{\operatorname{argmax}} \hat{P}(v_j|d) \prod_i \hat{P}(a_i|v_j, d) = \underset{v_j \in V}{\operatorname{argmax}} P(v_j|d) \prod_i P(a_i|v_j, d)$$

Naive Bayes Remarks

2. **Problem with attribute values missing in training examples.**

What if none of the training instances with target value v_j have attribute value a_i?

Then

$$\hat{P}(a_i|v_j, d) = 0$$

and so....

$$\hat{P}(v_j|d) \prod_i \hat{P}(a_i|v_j, d) = 0$$

But is v_j really an impossible classification value?

Naive Bayes Remarks

Typical solution is Bayesian estimate for $\hat{P}(a_i|v_j)$

$$\hat{P}(a_i|v_j) \leftarrow \frac{n_c + mp}{n + m} \quad \textit{instead of} \quad \frac{n_c}{n}$$

where

- n is number of training examples for which $v = v_j$,
- n_c is number of examples for which $v = v_j$ and $a = a_i$
- p is prior estimate for $\hat{P}(a_i|v_j)$ (e.g., uniform probability $1/|a_i|$)
- m is weight (integer) given to prior
- m called *equivalent sample size*: interpretable as augmenting the n examples by m virtual examples distributed as p.

Ex: $m = 10$, $|a_i| = 5$, $p = \frac{1}{5}$, $n_c = 0$, $n = 20$, $\hat{P}(a_i|v_j) = \frac{0+10*1/5}{20+10} = \frac{1}{15}$

Example: Learning to Classify Text

- Learn which news articles are of interest
- Learn to classify web pages by topic
- Learn spam messages

Naive Bayes is among most effective algorithms

What attributes shall we use to represent text documents?

Learning to Classify Text

Target concept *Interesting*? : $Document \rightarrow V = \{+, -\}$

1. Represent each document by a vector *doc* of words
   - one attribute per word position in document
2. Learning: Use training examples to estimate
   - $P(+)$
   - $P(-)$
   - $P(doc|+)$
   - $P(doc|-)$

doc: input vector of words (each word is an attribute value)

Note: simplified notation without the conditioning element of the examples (d) as in $\text{argmax}_{v_j \in V} P(a_1, a_2 \ldots a_n | v_j, d) P(v_j | d)$

Learning to Classify Text

Naive Bayes conditional independence assumption

$$P(doc|v_j) = \prod_{i=1}^{length(doc)} P(a_i = w_k | v_j)$$

where $P(a_i = w_k | v_j)$ is probability that word in position i of *doc* is w_k, given v_j

One more **assumption**: $P(a_i = w_k | v_j) = P(a_m = w_k | v_j)$, $forall\ i, m$
= The prob of a word in a certain position is independent from the position

$\Rightarrow$ thus consider only $P(w_k | v_j)$!

Learning to Classify Text

LEARN_NAIVE_BAYES_TEXT(*Examples*, V)

1. *collect all words and other tokens that occur in Examples*:
   $\Rightarrow$ *Vocabulary* $\leftarrow$ all distinct words and other tokens in *Examples*
2. *calculate the required* $P(v_j)$ *and* $P(w_k|v_j)$ *probability terms*:
   $\Rightarrow$ For each target value v_j in V do
   - $docs_j \leftarrow$ subset of *Examples* for which the target value is v_j
   - $P(v_j) \leftarrow \frac{|docs_j|}{|Examples|}$
   - $Text_j \leftarrow$ single document concatenating all members of $docs_j$
   - $n \leftarrow$ total number of words in $Text_j$ (counting duplicate words)
   - for each word w_k in *Vocabulary* do

     $n_k \leftarrow$ number of times word w_k occurs in $Text_j$

     $P(w_k|v_j) \leftarrow \frac{n_k+mp}{n+m} = \frac{n_k+1}{n+|Vocabulary|}$ with $m = |Vocaubolary|$ and $p = \frac{1}{m}$

Learning to Classify Text

CLASSIFY_NAIVE_BAYES_TEXT(*Doc*)

- *positions* $\leftarrow$ all word positions in *Doc* that contain tokens found in *Vocabulary* (if *doc* contains other words, they are ignored)
- Return v_{NB}, where

$$v_{NB} = \underset{v_j \in V}{\operatorname{argmax}} P(v_j) \prod_{i \in positions} P(w_i|v_j)$$

Twenty NewsGroups

Task: classify new documents in 20 classes of newsgroups ($|V| = 20$)

Training: 1000 training documents from each of 20 newsgroups.

Results: Naive Bayes: 89% classification accuracy

Random Guessing achieves accuracy of $\sim 5\%$

Filtering: the most "x" frequent words are removed (e.g., "the", "a", "of") as they don't give classification information. Same for rare words.